中山大学计算机学院

人工智能实验报告（第一次）

课程名称：Artificial Intelligence

教学班级	超算班	专业	信息与计算科学
学号	21307261	姓名	王健阳

一，实验题目

探究如何用 python 实现`Dijkstra`算法

二，实验内容

> **算法原理**

首先确定图中的某个源点`u`作为出发点，顶点集 `V` 分为集合 `S` 与`V - S`。初始时`S`中只有顶点`u`。设立辅助数组`DistTo`记录每个顶点到u的最短距离，`pq`则用来存放当前所有需要被考虑进去的顶点与其相应的最短距离，并全部初始化为`-1`。`DistTo[u]`初始化为整型最大值，其他初始化为正无穷。接下来便开始循环：每次从`DistTo`中取出（具体的操作就是将其索引处的`pq`值改为`-1`）对应数值最小的那个数，依次对其相连的边进行松弛，直到`pq`数组全部为`-1`时则停止循环。

松弛顶点指的是遍历该顶点`v`的每一条边并访问其终点`u`的对应的`DistTo`值，如果`DistTo[u] > DistTo[v] + weight`，则更新`DistTo[u]`的值，并将其写入`pq[u]`中去。

> **伪代码**

```
class Dijkstra:
    procedure __init__(graph, start)
        DistTo = {[VertexNames], [INTMAX]*VertexNums}
        EdgeTo = {[VertexNames], [EmptyEdge]*VertexNums}
        pq = {[VertexNames], [-1]*VertexNums}

        DistTo[start], pq[start] = 0, 0
        while !Finish(pq):
            v = delMin(pq).index
            relax(graph, v)

    procedure relax(graph, v)
        for edge in graph[v]:
            if DistTo[edge.end] > DistTo[v] + edge.weight:
                DistTo = DistTo[v] + edge.weight
                EdgeTo[v] = edge
                pq[v] = DistTo[v]

    procedure DistanceTo(v)
        return DistTo[v]
```

```
    procedure PathTo(v)
         p = v
        while true:
            path.add(EdgeTo[p].end)
            p = EdgeTo[p].start
            if p == start:
                path.add(p)
                return path.reverse()
```

> **关键代码展示**

```
class Dijkstra:
    def __init__(self, graph: Graph, start: str):
        self.check = ''
        self.DistTo = dict(zip(graph.NameList, [sys.maxsize] *
len(graph.NameList)))
        self.EdgeTo = dict(zip(graph.NameList, [Edge()] * len(graph.NameList)))
        self.pq = dict(zip(graph.NameList, [-1] * len(graph.NameList)))

        self.DistTo[start], self.pq[start] = 0, 0
        while not IsFinish(self.pq):
            min, min_city = sys.maxsize, 0
            for key in self.pq.keys():
                if self.pq[key] != -1 and self.pq[key] < min:
                    min, min_city = self.pq[key], key
            self.pq[min_city] = -1
            self.relax(graph, min_city)

    def relax(self, graph: Graph, v: str):
        for edge in graph.adj[v].edgeList:
            if self.DistTo[v] + edge.weight < self.DistTo[edge.end]:
                self.DistTo[edge.end] = self.DistTo[v] + edge.weight
                self.EdgeTo[edge.end] = edge
                self.pq[edge.end] = self.DistTo[edge.end]
```

> **创新点 & 优化**

在控制台输出最短路径的同时也输出了路径的累加过程，将计算出最短路径的过程可视化。对于用户输入添加了检验机制，增加了程序的容错性，并将错误之处反馈给用户（哪个地方输入有误）

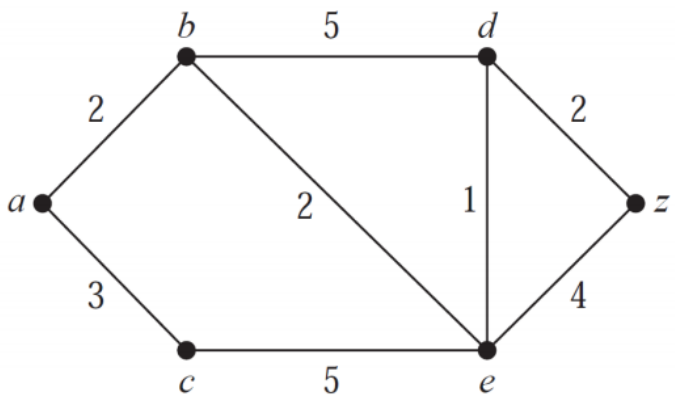
三，实验结果及分析

算法结果展示实例

> **实验1-1 最短路径问题**

- 样例

```
6 8
a b 2
a c 3
b d 5
b e 2
c e 5
d e 1
d z 2
e z 4
a z
```



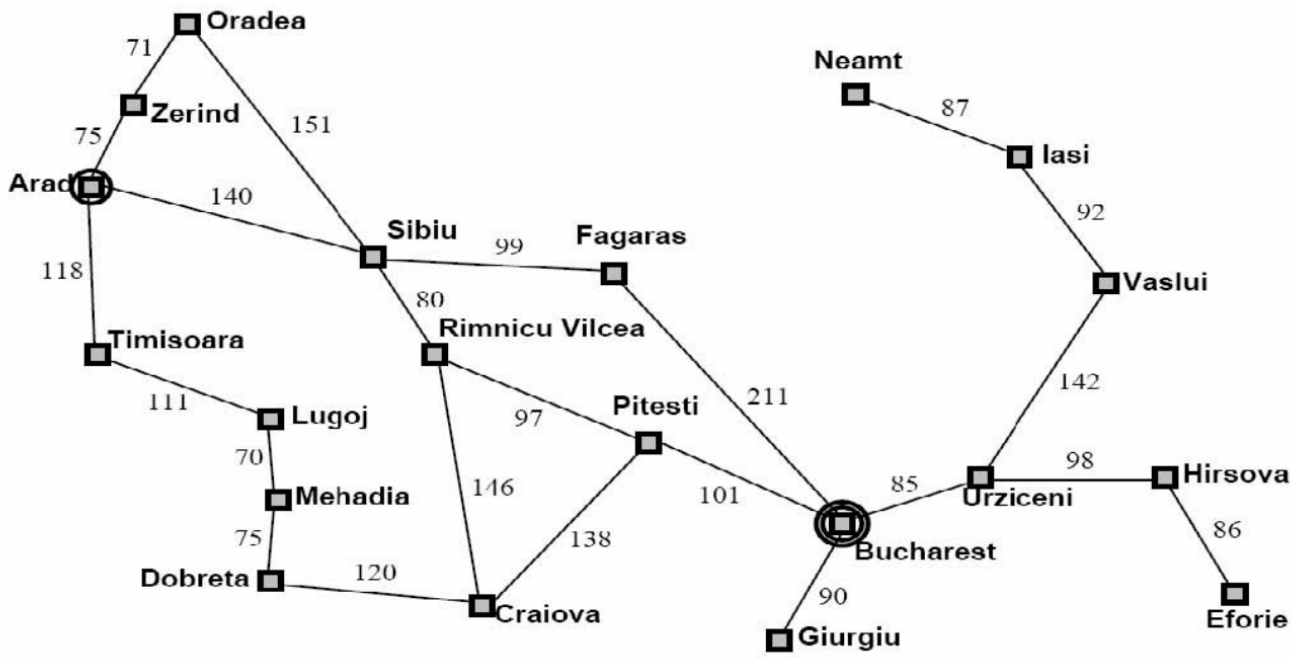
主函数代码

```
from source import *


def init():
    edges = []
    vertex_list = set()
    # 输入顶点数和边数
    (m, n) = input().split()
    for ii in range(int(n)):
        # 每行输入起终点以及权重
        (start, end, weight) = input().split()
        # 更新vertex_list集合内的值
        vertex_list.update({start, end})
        edges.append([start, end, int(weight)])
    return edges, list(vertex_list)


(edges, vertex_list) = init()
# 图初始化
graph = Graph(len(vertex_list), vertex_list)
# 依次插入每一条边
for edge in edges:
    graph.AddEdge(edge[0], edge[1], edge[2])
(st, ed) = input().split()
solution = Dijkstra(graph, st)
print('----------------------')
print(solution.DistanceTo(ed))
print('->'.join(solution.PathTo(ed)))
```

实验结果

```
C:\Users\王健阳\AppData\Lo
6 8
a b 2
a c 3
b d 5
b e 2
c e 5
d e 1
d z 2
e z 4
a z
---------------------
7
a->b->e->d->z
```

实验1-2 罗马尼亚旅行问题



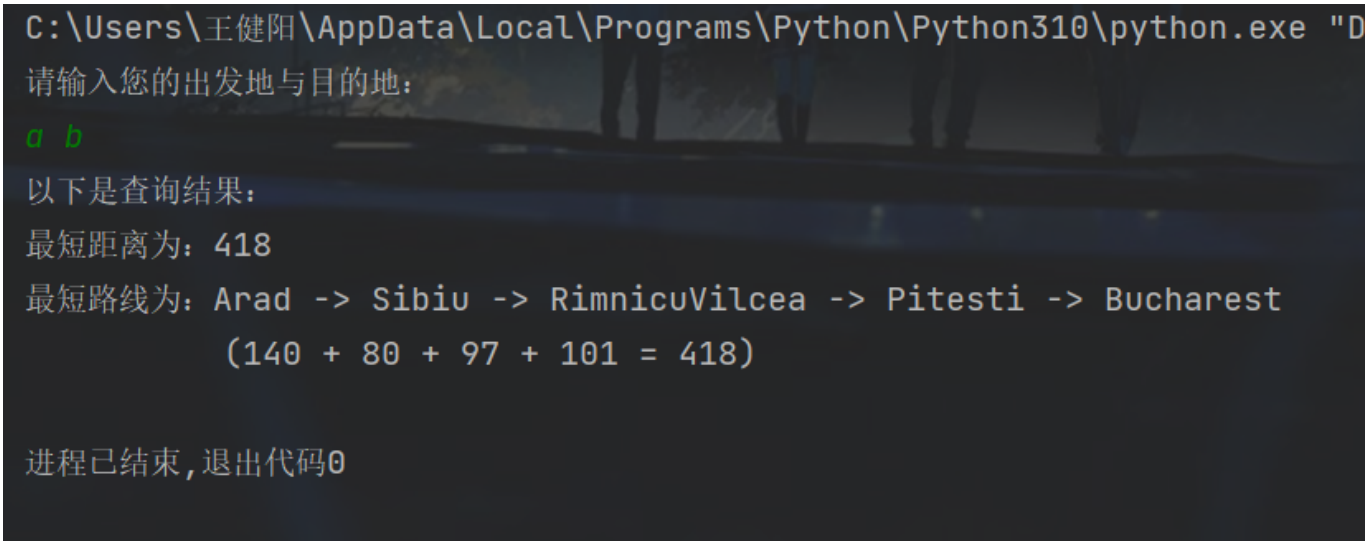
主函数代码

```
import functions

solution = None
# 导入数据，生成城市列表并初始化无向图
graph = functions.GraphInit()
file = open("记录日志.txt", 'r+')
ll = file.read()
if len(file.read().split('\n')) >= 10:
    file.truncate(0)
file.write('--------------------------------\n->')
```

```
address, destination = input('请输入您的出发地与目的地: \n').split()
# 进行模糊匹配
address, destination = functions.CityMatch(address.title()),
functions.CityMatch(destination.title())
if address == 'false' or destination == 'false':
    output = ('出发地' if address == 'false' else '目的地') + '输入错误，请再次检查'
    print(output)
    file.write(output + '\n')
else:
    solution = functions.Dijkstra(graph, address)
    distance = '最短距离为: ' + str(solution.DistanceTo(destination)) + '\n'
    path = '最短路线为: ' + ' -> '.join(solution.PathTo(destination)) + '\n
'
    # check字符串单纯用来展示最短路径的累加过程
    check = '(' + solution.check + ' = ' + str(solution.DistanceTo(destination)) +
')'
    # 将输出写入日志文件中
    file.write(distance + '\n  ' + path + '\n')
    # 直接输出内容反馈给用户
    print('以下是查询结果: 、')
    print(distance)
    print(path + '\n          ' + check)
# 关闭文件
file.close()
```

实验结果

From Arad to Bucharest :



From Fagaras to Dobreta :

```
C:\Users\王健阳\AppData\Local\Programs\Python\Python310\python.exe "D:/
请输入您的出发地与目的地：
f d
以下是查询结果：
最短距离为：445
最短路线为：Fagaras -> Sibiu -> RimnicuVilcea -> Craiova -> Dobreta
          (99 + 80 + 146 + 120 = 445)

进程已结束,退出代码0
```

From Mehadia to Sibiu :

```
C:\Users\王健阳\AppData\Local\Programs\Python\Python310\python.exe "D
请输入您的出发地与目的地：
m, s
以下是查询结果：
最短距离为：421
最短路线为：Mehadia -> Dobreta -> Craiova -> RimnicuVilcea -> Sibiu
          (75 + 120 + 146 + 80 = 421)

进程已结束,退出代码0
```

日志文件展示:

```
--------------------------------
->最短距离为：418
 最短路线为：Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
--------------------------------
->最短距离为：445
 最短路线为：Fagaras -> Sibiu -> RimnicuVilcea -> Craiova -> Dobreta
--------------------------------
->最短距离为：421
 最短路线为：Mehadia -> Dobreta -> Craiova -> RimnicuVilcea -> Sibiu
--------------------------------
->最短距离为：445
 最短路线为：Fagaras -> Sibiu -> RimnicuVilcea -> Craiova -> Dobreta
--------------------------------
->最短距离为：418
 最短路线为：Arad -> Sibiu -> RimnicuVilcea -> Pitesti -> Bucharest
```

四，思考题

```python
# 1
print(3 + 5)
print(12 - 4)
print(2 * 4)
print(int(16 / 2))
```

```python
# 2
famous_person = "Albert Einstein"
message = "A person who never made a mistake never tried anything new."
print(famous_person + " once said, \"" + message + "\"")
```

```python
# 3
s = "\n\tSchool of Computer Science and Engineering \n"
print(s)
print(s.lstrip())
print(s.rstrip())
```

```
print(s.strip())
print("--------------------------")
ss = s.strip()
print(ss.upper())
print(ss.lower())
print(ss.title())
```

```
# 4
alien_color = 'green'
point = 0
if alien_color == 'green':
    point = 5
elif alien_color == 'yellow':
    point = 10
elif alien_color == 'red':
    point = 15
print("Player gets " + str(point) + " points")
```

```
# 5
sum = 0
for ii in range(1, 101):
    sum += ii if ii % 2 == 0 else 0
print(sum)
```

```
# 6
import random

ans = random.randint(0, 10)
print(ans)
count = 1
guess = int(input())
while guess != ans:
    if guess > ans:
        print("猜大了")
    else:
        print("猜小了")
    guess = int(input())
    count += 1
print("一共猜了" + str(count) + "次")
```

```
# 7
sum = sum([ii for ii in range(1, 101)])
print(sum)
```

```python
# 8
import numpy as np

mat1 = np.array([[1, 2, 1], [2, 1, 1], [1, 1, 2]])
mat2 = np.array([[2, 3, 3], [3, 2, 3], [3, 3, 2]])

print(mat1 * mat2)
```

```python
# 9
CityList = ['Beijing', 'Shanghai', 'Guangzhou', 'Chongqing', 'Chengdu', 'Wuhan',
'Guiyang', 'Kunming']
cities = {"Beijing": {'country': 'China', 'population': 2000_0000, 'fact': '中国的
政治中心'}, 'Guangzhou': {'country': 'China', 'population': 1000_0000, 'fact': '中
国的养老中心'}, 'Shanghai': {'country': 'China', 'population': 1500_0000, 'fact':
'中国的经济中心'}}
print(cities)
```

```python
# 10
def show_magician(manelist: list):
    for ii in manelist:
        print(ii)

def make_great(namelist: list):
    for ii in range(len(namelist)):
        namelist[ii] = 'the Great ' + namelist[ii]

def make_great_2(namelist: list):
    lst = []
    for ii in namelist:
        lst.append('the Great ' + ii)
    return lst


# a)
lst = ['magician_A', 'magician_B', 'magician_C', 'magician_D', 'magician_E',
'magician_F', 'magician_G']
show_magician(lst)

# b)
#make_great(lst)
#show_magician(lst)

# c)
show_magician(make_great_2(lst))
show_magician(lst)
```

```python
# 11
# a)
class resturant():
    def __init__(self, resturant_name, cuisine_type):
        self.resturant_name = resturant_name
        self.cuisine_type = cuisine_type
        # b)
        self.number_served = 0

    def describe_resturant(self):
        print('resturant name is ' + self.resturant_name)
        print('our cuisine type is ' + self.cuisine_type)

    def open_resturant(self):
        print('我们正在营业中，欢迎光临！ ')

    # b)
    def set_number_served(self, num: int):
        self.number_served = num

    def increment_number_served(self, num: int):
        self.number_served += num


# c)
class IceCreamStand(resturant):
    def __init__(self, resturant_name, cuisine_type, flavors):
        super().__init__(resturant_name, cuisine_type)
        self.flavours = flavors

    def show(self):
        for ii in self.flavours:
            print(ii)


kfc = resturant('KFC', '开封菜')
print(kfc.number_served)
kfc.set_number_served(100)
print(kfc.number_served)
kfc.increment_number_served(2000)
print(kfc.number_served)

flavours = ['chocolate', 'milk', 'banana', 'blueberry', 'pineapple']
DairyQueen = IceCreamStand('DairyQueen', 'IceCream', flavours)
DairyQueen.describe_resturant()
DairyQueen.open_resturant()
DairyQueen.show()
```

```python
# 12
import requests
from bs4 import BeautifulSoup
```

```
url = 'https://gutenberg.org/cache/epub/70165/pg70165-images.html'
headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36
Edg/110.0.1587.57'}
# Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/110.0.0.0 Safari/537.36 Edg/110.0.1587.57

strhtml = requests.get(url, headers=headers)  # Get方式获取网页数据
soup = BeautifulSoup(strhtml.text, 'lxml')  # 将请求到的数据解析为lxml格式
info = soup.select('body')  # 筛选数据，这里需要加一个参数

word_list = []
count = 0
with open("12.txt", 'w', encoding='utf-8') as file:
    for item in info:
        ss = item.get_text().lower()
        count += ss.count('the')
        word_list.extend(ss.split())
        file.write(item.get_text())
print('大约一共有 ' + str(len(word_list)) + ' 个单词')
print('大约一共有 ' + str(count) + '个单词the')

print("------------------------------------------------------------")
for item in info:
    print(item.get_text())
```

```
# 13
dit = {1: 1, 2: 2, 3: 3}

try:
    print(dit[1])
except NameError:
    print('Name wrong!')
except KeyError:
    print('Key wrong!')
else:
    print('else!')
finally:
    print('finally!')

try:
    print(dit[4])
except NameError:
    print('Name wrong!')
except KeyError:
    print('Key wrong!')
else:
    print('else!')
finally:
    print('finally!')
```

五，参考资料

1.实验python基础pdf

2.CSDN